

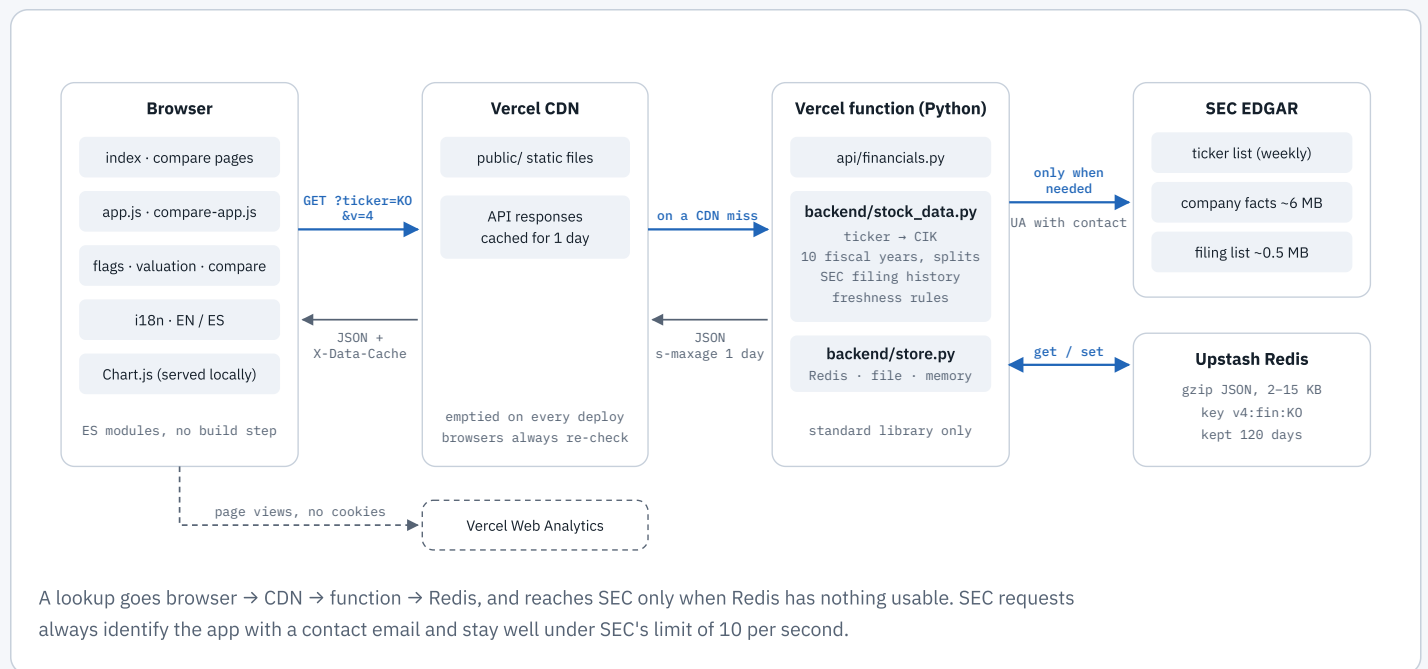
10-Year Stock Value Analysis

Two plain HTML and JavaScript pages (single-company results and a side-by-side compare page) on Vercel's CDN, one small Python function, SEC EDGAR as the only data source, and Upstash Redis so each company is fetched once. Everything is linted and tested on every push, the results are published as a public report, and a scheduled check watches the live site.

stock-value-analysis.vercel.app stock-trend-test-report.vercel.app github.com/dipak-nehe/stock-trend-analyzer

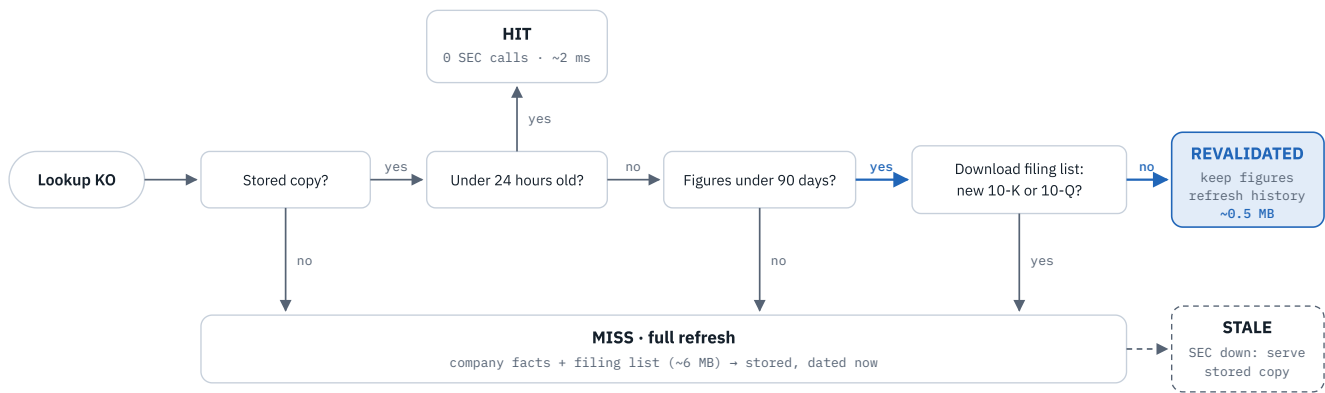
1. How a lookup travels

The blue path is a ticker lookup. Most repeat lookups stop at the CDN or at Redis and never reach SEC.



2. How stored results stay fresh

Annual figures change once a year and the share count once a quarter, but red flags can appear any day. So after 24 hours the function downloads only the small filing list, and fetches the large company facts only when a new report exists.



The blue path is the everyday case after the first day: one small download, and red flags such as late filings still appear within a day. A new annual or quarterly report, or figures older than 90 days, force a full refresh.

6.17 MB · 1.17 s
MSFT, first lookup (MISS)

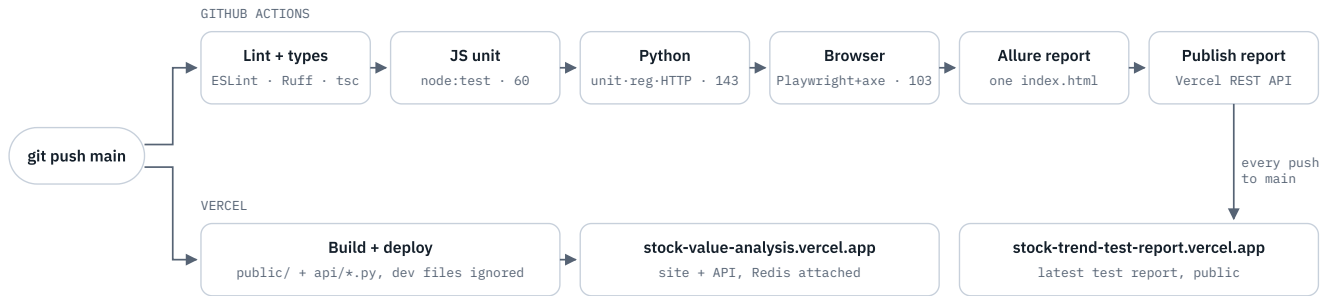
0.49 MB · 0.31 s
MSFT, re-check after 24 h
(REVALIDATED)

~2 ms
stored result (HIT), 0 SEC calls

2.2 KB
MSFT result as stored, gzip-compressed

3. How changes ship

One push to main runs every test layer and deploys the site. Vercel builds the site; GitHub Actions tests it and publishes the test report.



Tests run in each step even if an earlier step failed, so the report always covers every layer. Deploys need commits authored with the email linked to the GitHub account; Vercel blocks others. Outside this flow, a scheduled GitHub Action checks both live pages and a fresh API answer every 6 hours, and Dependabot opens weekly grouped update pull requests that go through the same tests.

TEST LAYER	TOOL	WHAT IT PROVES	TESTS
JavaScript unit	node:test	Formatting, trend labels, every red-flag rule, checklists, comparison rules, translations	60
Unit + storage	pytest	Splits, restatements, currency, debt and liabilities, freshness rules, simulated Redis	78
Regression	pytest	Saved real filings match published figures for 2021–2025	42
HTTP	pytest	Local server and Vercel function behave the same; source files never served	23
End-to-end	Playwright	Search, tabs, glance card, price, Home button, compare page (marks, prices, swap, deep links), Spanish, errors, phone layout	72

TEST LAYER	TOOL	WHAT IT PROVES	TESTS
Accessibility	axe-core	WCAG 2.2 AA in light and dark, both languages, compare page, keyboard only	31
Total		Plus ESLint, Ruff and TypeScript checks before any test runs	306

Where each piece lives

PART	FILES	JOB
Pages	public/index.html, compare.html, styles.css	Results page and compare page with shared styles; static text tagged for translation
Front-end logic	public/js/*.js	Pure modules (rules, views, formatting) including compare.js, plus app.js, compare-app.js, page.js (shared) and charts.js, which touch the page
Translations	public/js/strings/en.js, es.js	English and Spanish (Spain) text, same keys and placeholders
API	api/financials.py	Vercel function: one GET endpoint
Data layer	backend/stock_data.py	SEC fetching, normalisation, filing history, freshness rules
Storage	backend/store.py	Redis (live), files (local), memory (tests)
Local server	server.py	Same API for development, serves public/
Deploy settings	vercel.json, .vercelignore	Output folder, bundled modules, security headers, what not to upload
CI	.github/workflows/tests.yml	Checks, tests, Allure report, report publishing
Live-site check	.github/workflows/health.yml	Both pages and a fresh API answer every 6 hours; GitHub emails on failure
Dependency updates	.github/dependabot.yml	Weekly grouped updates for Python, npm and Actions, plus security alerts
Lint settings	eslint.config.js, ruff.toml, tsconfig.json	JavaScript lint, Python lint, JavaScript type check